

1. Objective

Design and implement a simplified backend system for a social media platform. The goal is to demonstrate a clear understanding of RESTful API principles, the DTO (Data Transfer Object) pattern, and basic CRUD (Create, Read, Update, Delete) operations using Spring Boot.

2. Requirements

Part A: User Management Service

You need to manage the identity of the people using your platform. This service must operate independently of any other feature.

Entity Properties: id (Long), fullName (String), email (String), and bio (String).

Constraint: The email must be unique.

Data Flow:

Input: Use a UserRequestDTO for creating a user.

Output: Use a UserResponseDTO when fetching data (ensure the id is visible, but internal system flags are hidden).

Part B: Content Management Service (Posts)

You need to manage the "Feed." For this stage, assume the post is anonymous or the author's name is just a simple text field.

Entity Properties: id (Long), authorName (String), content (String), and createdAt (LocalDateTime).

Constraint: The content cannot be empty and should not exceed 280 characters.

Data Flow:

Input: Use a PostRequestDTO (only authorName and content).

Output: Use a PostResponseDTO (include the createdAt timestamp).

3. Technical Deliverables

Persistence: Store all data in a database Postgresql using Spring Data JPA.

API Endpoints:

POST /users & POST /posts – To save new data.

GET /users/{id} & GET /posts – To retrieve data.

PUT /users/{id} – To update a user's bio.

DELETE /posts/{id} – To remove a post.

Validation: Ensure that if a user sends invalid data (like an empty post), the API returns a proper error status.

4. Success Criteria

No Relationships: There should be no @ManyToOne or @OneToMany annotations in your code yet.

Layer Separation: Your code must clearly separate Controller (web), Service (logic), Repository (database), and DTO (data transfer).

Functionality: A user should be able to create an account and then create a post as two separate, successful actions.

5. Bonus Challenge

Implement a Soft Delete for the Posts. Instead of removing the post from the database, add a boolean active field. When the "Delete" API is called, simply change active to false and ensure that GET /posts only shows "active" content.